



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 15

Resumen del curso

Ciclo 2026-1

Agenda

1. Introducción
2. TAD
3. Files
4. Estructuras estáticas
5. Métodos de ordenamiento
6. Métodos de búsqueda
7. Listas enlazadas
8. Pilas
9. Colas
10. Arboles
11. Arboles B
12. Grafos
13. Referencias

1. Introducción

(Conceptos fundamentales)

(Errores ortograficos problemas de configuracion)

Datos

- **Dato**

- Es la representacion (basica) simbolica (numerica, alfabetica) de un atributo o variable cuantitativa. Los datos describen hechos empiricos, sucesos y entidades [<http://es.wikipedia.org/wiki/Dato>]

- **Informacion.**

- Es el dato con un valor agregado (datos procesados y relacionados)
 - Es el resultado de organizar y procesar estos datos para que tengan significado logico y sea utiles.
 - *Felipe tiene 15 de calificacion.* A partir de “Felipe” y “15”, la informatica procesa los datos para convertirlos en informacion.

Instrucción

- Una **Instrucción** es una operación que produce una *transformación* sobre los datos o produce una modificación en el flujo de control en la ejecución del programa
- Una instrucción es una operación que produce una transformación del estado en que se encuentra la ejecución del programa.

Estado

- Se denomina ***estado*** en un modelo de cómputo a un punto en la ejecución de programa en dicho modelo.
- En el modelo de Von Newman, corresponde al valor de las variables, constantes y la instrucción del programa que corresponde a dicho punto de la ejecución, entonces: “Es la foto de la computadora en un momento determinado”

Algoritmo

Es el diseno ***abstracto*** y ***conceptual*** de la solucion de un problema. Es un conjunto de pasos detallado para realizar una tarea.

- **Abstracto.** Implica ignorar detalles NO esenciales para resolucion de problemas y centrarse solo en aspectos relevantes: entrada, proceso y salida.
 - Permite manejar la complejidad
 - Algoritmo de busqueda: define que debe recibir una colección de datos y el dato a buscar y dara una respuesta Si o No esta.
- **Conceptual.** Representa un modelo mental o logico de la solucion, una serie de pasos o reglas bien definidas y ordenadas que describen la logica del proceso.
 - Facilita la comunicacion de la solucion a otros sin necesidad de conocer el codigo solucion. (receta de cocina)
 - Sirve como puente entre la idea de la aolucion y la implementacion real en el codigo funcional.

Programa

Es la implementacion de un algoritmo en un lenguaje de programacion determinado como: C, C++, Java, Python, Rus, ...

TAD (Tipo abstracto de dato)

Es un modelo matematico que describe un conjunto datos y las operaciones que se pueden realizar sobre ellos de forma independiente de su implementacion.

- **Objeto abstracto.** Representaion logica de los datos del TAD
- **Invariante.** Es la propiedad que debe cumplir en todo momento el TAD.
- **Operaciones.** Se definen sin detallar su implementacion interna y pueden ser constructoras, desructuras, analizadores, modificadores y persistencia.
- **Especificacion de las operaciones.** Se define por su comportamiento, independientemente de la implementacion..

Especificación de operaciones

- Especificar un algoritmo es describir **qué** es lo que hace a partir de detallar cuál es el **estado posterior a su ejecución** en función del **estado anterior** a la misma.

PRE = {precondición}

PROCESO

POS = {postcondición}

Especificación de operaciones

Ejemplo:

float divide (**float** x, **float** y)

//Toma dos números reales y retorna la división del primero entre el segundo

PRE = {y ≠ 0.0}

POS = { RETORNA (x/y)}

void sumaVector (**float** X[max], **int** dimX, **float** suma)

//Suma los elementos del vector y lo almacena en suma

PRE = {dimX>0 & dimX <max}

POS = {suma = suma elementos del vector posiciones entre [0, dimX-1]}

1. Estructura de datos

Estructura de datos

Las estructuras de datos son la forma en que se organizan, almacenan y gestionan los datos en un sistema informático

- Para que sean eficientes y se pueda acceder a ellos de manera rápida.
- Permiten a los programadores optimizar el rendimiento de las aplicaciones y son fundamentales para construir algoritmos.
- Se estudian como conceptos de programación y no sobre un lenguaje de programación y c/u de estas pueden hacer diferentes implementaciones.
- Una estructura de datos proporciona un formato específico para manejar datos abstractos y definir las relaciones entre ellos.

Estructura de datos: Utilidad

Es un elemento clave para construir software y sistemas informáticos.

Organización: Facilitan la organización lógica de grandes conjuntos de información compleja, haciéndolos más fáciles de entender y manejar tanto para los programadores como para los sistemas.

Eficiencia: La elección de la estructura de datos adecuada permite el uso de algoritmos más rápidos, optimizando el rendimiento del código informático.

Flexibilidad: Existen diferentes tipos de estructuras, cada una diseñada para un propósito específico, lo que permite a los desarrolladores elegir la herramienta más adecuada para cada tarea.

Estructura de datos: Tipos

- **Lineales.** Los elementos se organizan en una secuencia unos tras otro, donde cada elemento tiene un predecesor y sucesor únicos
 - Arreglos
 - Listas enlazadas
 - De acceso restringido**
 - Pilas
 - Colas
- **No lineales.** Los elementos no están en una secuencia única y pueden tener múltiples conexiones.
 - Árboles
 - Tablas Hash

2. TAD Tipo Abstracto de Datos

TAD (Tipo abstracto de dato)

Es un modelo matematico que describe un conjunto datos y las operaciones que se pueden realizar sobre ellos de forma independiente de su implementacion.

- **Objeto abstracto.** Representaion logica de los datos del TAD
- **Invariante.** Es la propiedad que debe cumplir en todo momento el TAD.
- **Operaciones.** Se definen sin detallar su implementacion interna y pueden ser constructoras, desructuras, analizadores, modificadores y persistencia.
- **Especificacion de las operaciones.** Se define por su comportamiento, independientemente de la implementacion..

2. TAD Definición

Definición formal:

TAD es una estructura algebraica compuesta por un conjunto de:

- ***Objetos abstractos***, que modelan elementos de la realidad, y
- ***Un Conjunto de operaciones***, los cuales simulan el ***comportamiento*** que el elemento tiene en la realidad del problema que se quiere solucionar.
- Independiente del lenguaje y de estructura de datos.

3. Formato del TAD

El TAD se define con un ***nombre***, un *formalismo* para expresar un ***objeto abstracto***, un ***invariante*** y un conjunto de ***operaciones*** sobre este objeto:

TAD < nombre >
< Objeto abstracto >
< Invariante del TAD >
< Operaciones >

4. Representación

OBJETO ABSTRACTO :

El TAD representa a un **objeto ABSTRACTO** :

- SIN establecer compromiso con ninguna estructura de datos ni lenguaje de programación.
- Debe permitir ***expresar las condiciones, relaciones, restricciones y operaciones*** de los elementos modelados sin necesidad de restringirse a una representación interna concreta.

5. Invariante

El objeto abstracto debe modelar un elemento posible de la realidad.

Establece una noción de **validez** para cada uno de los **objetos abstractos**, en términos de condiciones sobre su estructura interna y sus componentes.

Ejemplo:

El TAD conjunto y su notación: $A = \{ x_1, x_2, \dots, x_N \}$.

El invariante debe exigir que los x_i sean del mismo tipo de dato y que sean diferentes entre si para un objeto abstracto modele realmente un **Conjunto**.

2. TAD Grafo tipo TD

Objeto abstracto:

Grafo: (V, A) , $V = \{v_1, v_2, \dots, v_n\}$, $A = \{(v, w, p), \dots\}$

Invariante:

Para todo: $(v, w, p) \in A \implies v, w \in V \implies v \rightarrow w$ con peso $p \geq 0$

e)

Nombre	Dominio	Codominio
crearGrafo		Grafo
addVertice	Grafo x T D	Grafo
addArista	Grafo x ENTERO x ENTERO x ENTERO	Grafo
deleteArco	Grafo x ENTERO x ENTERO	Grafo
pesoArco	Grafo x ENTERO x ENTERO	ENTERO
mostrarVertice	Grafo x ENTERO	TD

3. Files (archivos)

3. Archivos

Introducción

- **Archivo.** Es una estructura de datos para almacenamiento secundario o un conjunto de bytes almacenado en un dispositivo de almacenamiento.
- La memoria principal (RAM) del computador es volátil, escasa y costosa.
- ***La memoria secundaria es permanente abundante y menos costosa.***
 - **Magnéticos:** HD, Cintas,
 - **Electrónicos;** USB, SSD (Solid State Drive), SSHD, mSATA (Serial Advanced Technology Attachment), SATAe, M.2, PCIe (Peripheral Component Interconnection Express), etc.
 - **Ópticos:** CD, DVD, Blu-ray

3. Archivos

Por su contenido en cada byte

- **Texto.** Están permitidos ciertos rangos de valores en cada byte, incluyen valores hexadecimal: 0x1A marca de fin de fichero. No es posible leer mas de un byte.
- **Binarios.** Están permitidos todos los valores para cada byte. El final de file se detecta de otro modo dependiendo del SO, generalmente es la longitud del file.

3. Archivos

Por el tipo de acceso. Es la tecnica que se utiliza para representar y almacenar registros en los archivos, veamos las dos mas usadas.

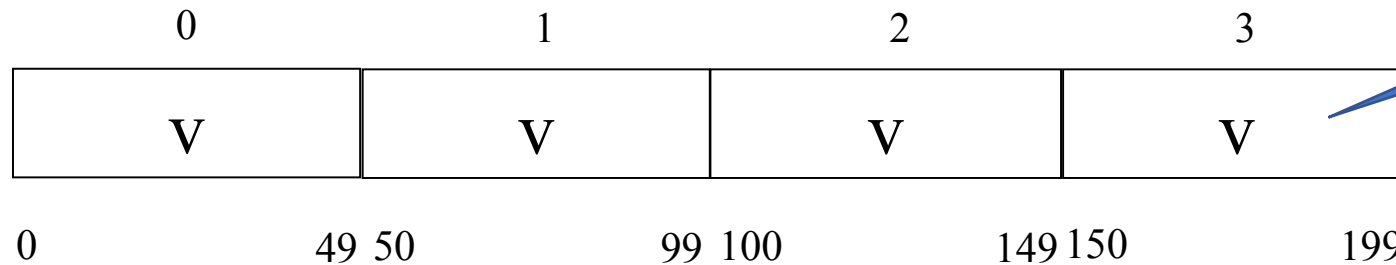
- **Secuencial.** Se accesar en forma secuencial a los elementos del archivo.
- **Directo:** Permite el acceso al registro deseado directamente mediante la formula :
*direccionFisica = direccionLogica*tamanoRegistroEnBytes.*

Fundamentos: Sea el registro de alumnus.

- REGISTRO ALUMNO
 - ENTERO codigo // 4 bytes
 - CHARACTER nom[42] // 42
 - REAL nota // 4
- FIN_REGISTRO // 50 bytes

3. Archivos

- REGISTRO ALUMNO
 - ENTERO codigo // 4 bytes
 - CHARACTER nom[42] // 42
 - REAL nota // 4
- FIN_REGISTRO
- $dirFisica = dirLogica * TamanoRegistroEnBytes$



Direccion logica

Registros

Direccion fisica

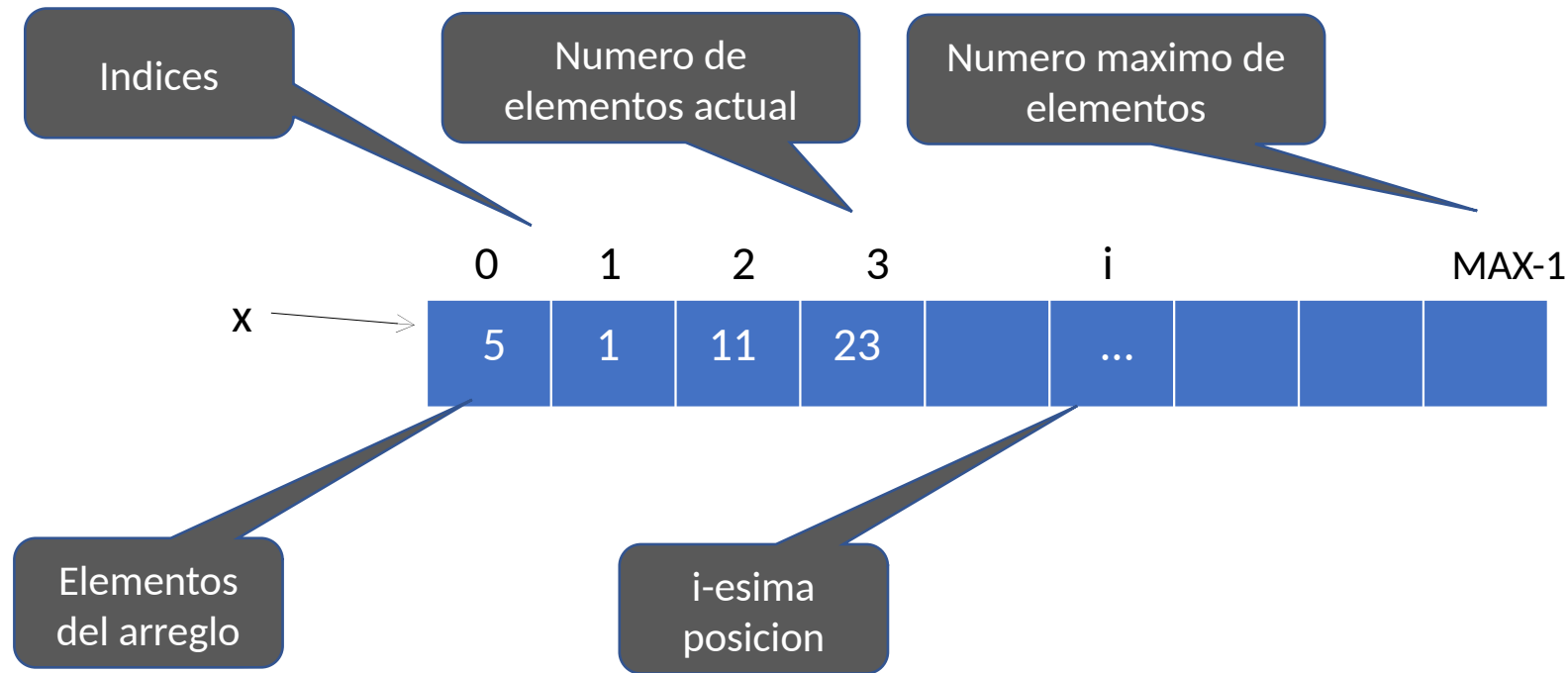
4. Estructuras estáticas (estructura básica)

4. Estructuras estáticas

- **Arreglo.** Es una estructura estática, contigua, homogénea, ordenada y finita de elementos.
- Son estructuras estáticas básicas que nos permiten implementar otras estructuras como pilas, colas, listas, arboles, grafos, etc

4. Estructuras estáticas

- **Arreglo.** Colección de elementos homogéneos almacenados en direcciones de memoria contiguas.



4. Estructuras estáticas

- **Arreglo.** Es una estructura estática, contigua, homogénea, ordenada y finita de elementos.
 - **Estática.** Tiene un numero máximo de elementos y debe ser declarado antes de definirse el arreglo.(MAX, MAXF, MAXC, ...)
 - **Contigua.** Elementos uno a continuación de otro elemento.
 - **Homogénea.** Todos sus elementos son del mismo tipo.
 - **Ordenada.** Ocupan un determinado lugar en la estructura y podemos citar como primer, segundo, ..., enésimo elemento.
 - **Finita.** Pues tiene un numero finito de elementos.

4. Estructuras estáticas

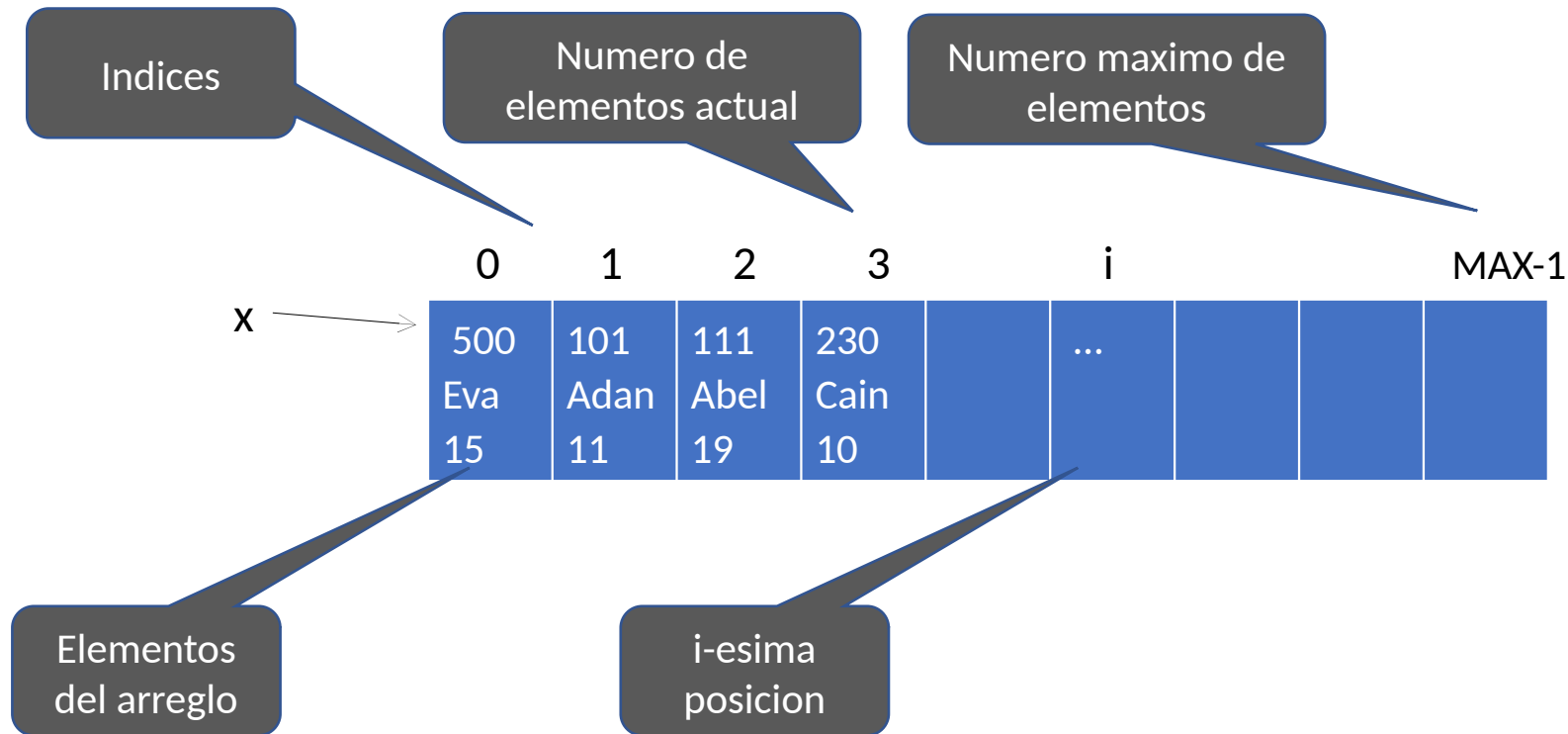
- **Arreglo.** Existe varios tipos de arreglos.
 - **Unidimensionales.** Denominados también vectores, se accesa a través de un nombre y un subíndice.
 - **Bidimensionales.** Denominados también matrices, se accesa a través de su nombre y dos subíndices, el primero denominado fila y el segundo columna.
 - **Multidimensionales.** Se refiere a arreglos de mas de dos dimensiones por ejemplo de subíndices i, j, k . En realidad es un arreglo con múltiples vectores.

4. Estructuras estáticas

- **Ventajas.**
 - **Tiempo acceso constante.** El tiempo de acceso al primer o ultimo elementos es el mismo.
 - Es ED contigua de datos cada índice apunta directamente al espacio donde se encuentra el dato, esto es, acceder directamente a ello:
 - Por lo anterior, el tiempo de acceso a los elementos es constante.
- **Desventajas.**
 - **Es volátil**
 - **Inserción/eliminación.** De elementos es costoso pues tiene que desplazar elementos a la derecha o izquierda respectivamente .
 - **Finita.** Pues tiene un numero finito de elementos.
 - Es un problema determinar el tamaño adecuado, si es pequeño no nos sirve y si muy grande es un desperdicio de memoria.

4. Estructuras estáticas registro

- **Introducción.** Colección de elementos del mismo tipo compuesto (registro) almacenados en direcciones de memoria contiguas.



4. Estructuras estáticas

- **Registro.** Es un TIPO DE DATO estructurado formado por un conjunto de elementos de diferente de tipos de dato bajo la misma estructura.
 - Un registro almacena un conjunto de datos relacionados.
 - Los elementos de un registro se denominan ***campos***.
 - Cada uno de los ***campos*** pueden ser de tipo de dato predefinido, arreglos e incluso tipos de dato de tipo registro.
 - Los Registros extienden la utilidad de los lenguajes a nuevos tipos de dato de la realidad.
 - Al tipo registro también se le conoce como tipos de datos creado por el usuario.
 - Por ejemplo, puedo crear un tipo de dato ALUMNO cuyos campos son código, nombre y nota.
 - Como ALUMNO es un tipo de dato puedo crear un vector de tipo ALUMNO

4. Estructuras estáticas

- **Combinaciones.** Existe varios tipos combinaciones de registros con arreglos, veamos:.

- **Vector de Registros.** Podemos crear un vector de registros.
- **Registros con vectores.** Es un registro que tiene como campo a un vector.
- **Registros anidados.** Es un registro que tiene como campo a una variable de tipo de registro.

- **REGISTRO FECHA**

- ENTERO dd
- ENTERO mm
- ENTERO aaaa

- **FIN_REGISTRO**

- **REGISTRO ALUMNO**

- ENTERO cod
- **FECHA** fnac
- REAL p[10]

- **FIN_REGISTRO**

ALUMNO alu[MAX]

Registro
anidado

Registro con
vectores

Vector de
Registros

4. Estructuras estáticas

- **Registros.** Permiten *crear nuevos tipos datos* basados en tipos de dato predefinidos.
 - Como los campos pueden ser diferentes tipos, permite al usuario crear diferentes tipos de datos de la realidad.
 - El nuevo tipo de dato de tipo registro tiene las mismas prerrogativas de los tipos de dato predefinidos.
 - Podemos crear arreglos de registros obteniendo así todas las características de este tipo de estructura.

4. Estructuras estáticas

- **Ventajas.**

- Una estructura reserva espacio para todos sus elementos y se puede asignar valores individualmente.
- Permite crear nuevos tipos de datos.
- Permite desarrollar aplicaciones para resolver problemas de la realidad.

- **Desventajas.**

- No permite módulos de operaciones sobre los elementos campos de la estructura y si lo permite no hay costumbre en esta metodología, para eso está la POO.

5. Métodos de ordenamiento

5. Métodos de Ordenamiento

1. Introducción

- Es la operación de organizar un conjunto de elementos en algún orden o secuencia específica para datos numéricos o datos alfanuméricos de tipo de carácter.
- Las operaciones comunes de ordenamiento son listas de números, de archivos o de nombres en una agenda, por ejemplo.
- Es importante la operación de ordenamiento para optimizar otros algoritmos como los de búsqueda.

5. Métodos de Ordenamiento

- La ordenación es poner en orden ascendente o descendente con el propósito de clasificarlos para facilitar la manipulación de datos en un vector.

5. Métodos de Ordenamiento

- Los métodos de ordenamiento se clasifican en :
- **Ordenación interna.** Es el ordenamiento en memoria principal
 $O(n^2)$:
 - **Intercambio.** Burbuja, b con señal, b de sacudida. (n^2)
 - **Inserción.** Inserción, Inserción binaria
 - **Selección.** $O(n \log n)$. Algoritmos eficientes
 - **Método Shell**
 - **Método QuickSort**
 - **Método del montículo.**
- **Ordenación externa.** Es el ordenamiento de memoria secundaria. Los arboles B optimizan el acceso a disco.
 - **Intercalación**
 - **Ordenación**

6. Métodos de búsqueda

6. Métodos de búsqueda

1. Introducción

- La recuperación de la información es una de las operaciones más importantes.
- La búsqueda es la operación de encontrar la posición de un elemento o de un conjunto de elementos de un arreglo, tabla, lista o archivo.
- Existe diferentes métodos de búsqueda, el método de búsqueda depende de la estructura donde se encuentren almacenados los datos.

6. Métodos de búsqueda

- **Tipos**

- **Busqueda secuencial/lineal $O(n)$** . Funciona en cualquier lista.
- **Busqueda binaria $O(\log n)$** . Requiere de lista de elementos ordenada.
- ABB, AVL, B
- Tablas hash

6. Métodos de búsqueda

- **Desventajas**

- Tiene el inconveniente de consumo excesivo de tiempo de búsqueda de la localización del dato buscado.
- Cuando el dato buscado no se encuentra en el vector se verifica los n elementos del vector.; por estas razones no es el método más adecuado para un vector con gran número de elementos.

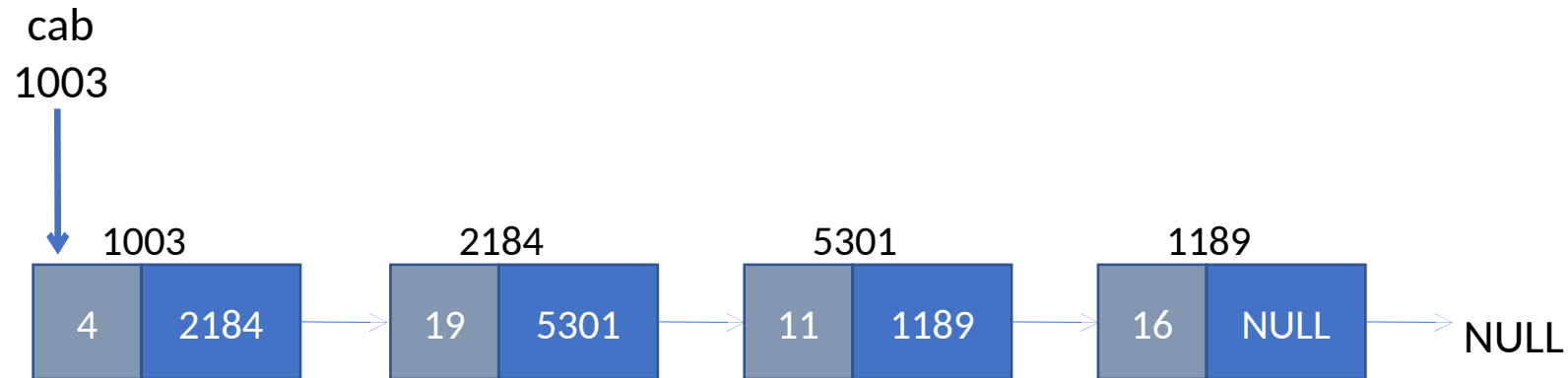
7. Listas enlazadas

7. Listas Enlazadas

1. **Introducción** . Secuencias de elementos donde cada uno apunta al siguiente. Son estructuras de datos fundamentales que pueden ser usadas para implementar otras estructuras de datos.
 - Fueron desarrolladas por Cliff Shaw y Herbert Simon en 1956
 - La “magia” de las estructuras enlazadas es dada por los apuntadores (pointers)
 - Los lenguajes imperativos u orientados a objetos como C, C++, Java, Python, ... disponen de referencias para crear listas enlazadas.

7. Listas Enlazadas

- Definición .** Es una estructura de datos lineal dinámica de un conjunto de datos homogéneos ligados entre si por una referencia al siguiente elemento, conformada de nodos donde cada nodo contiene la información que puede almacenar y un apuntador al siguiente nodo.



7. Listas Enlazadas

1. Características.

- Son estructuras dinámicas que se crean y se eliminan en tiempo de ejecución
- Pueden expandirse y contraerse fácilmente (inserciones y eliminaciones)
- Se almacena en posiciones de memoria no contiguas.
- El acceso a los elementos es secuencial.
- Recordemos la representación lógica varia con la representación física

7. Ventajas y Desventajas

1. Ventajas.

- Son dinámicas, no necesita saber el número de elementos a almacenar.
- Son estructuras flexibles
- Pueden expandirse y contraerse fácilmente

2. Desventajas.

- Por su acceso secuencial no son buenas para búsquedas en grandes cantidades de datos.
- Se necesita espacio adicional para almacenar los enlaces

7. Apliaciones

1. **Las listas simples y dobles.** Tienen características de avance hacia adelante, hacia adelante y hacia atrás, son ideales para:
 - **Visores de imagenes.** Permiten navegar hacia adelante y hacia atras.
 - **Gestores de reproduccion de musica.** Donde una cancion apunta a otra.
 - **Sistemas operativos.** Para rastrear de foma dinamica bloques de memoria libre y asignada

8. Pilas

Estructura de acceso restringido

8. Pilas

1. Definición. La pila es una estructura lineal de una secuencia de elemento de cierto tipo en donde ***se inserta y elimina elementos por uno de los extremos.***
 - Características
 - Estructura de acceso restringido
 - Es una estructura LIFO (last in first out) ultimo en entrar, primero en salir.
 - Es una colección de datos, se puede acceder por un extremo denominado ***Tope.***
 - En cada momento solo se tiene acceso al tope de la pila o sea al ultimo elemento apilado.
 - Es una estructura ordenada, podemos decir este es el primer, segundo, etc., elemento.

8. Pilas

1. Representación. Las pilas no son estructuras fundamentales o sea no están definidas como tales en los lenguajes, se puede hacer mediante estructuras:
 - **Estáticas.** Mediante arreglos
 - **Dinámicas.** Mediante listas enlazadas.

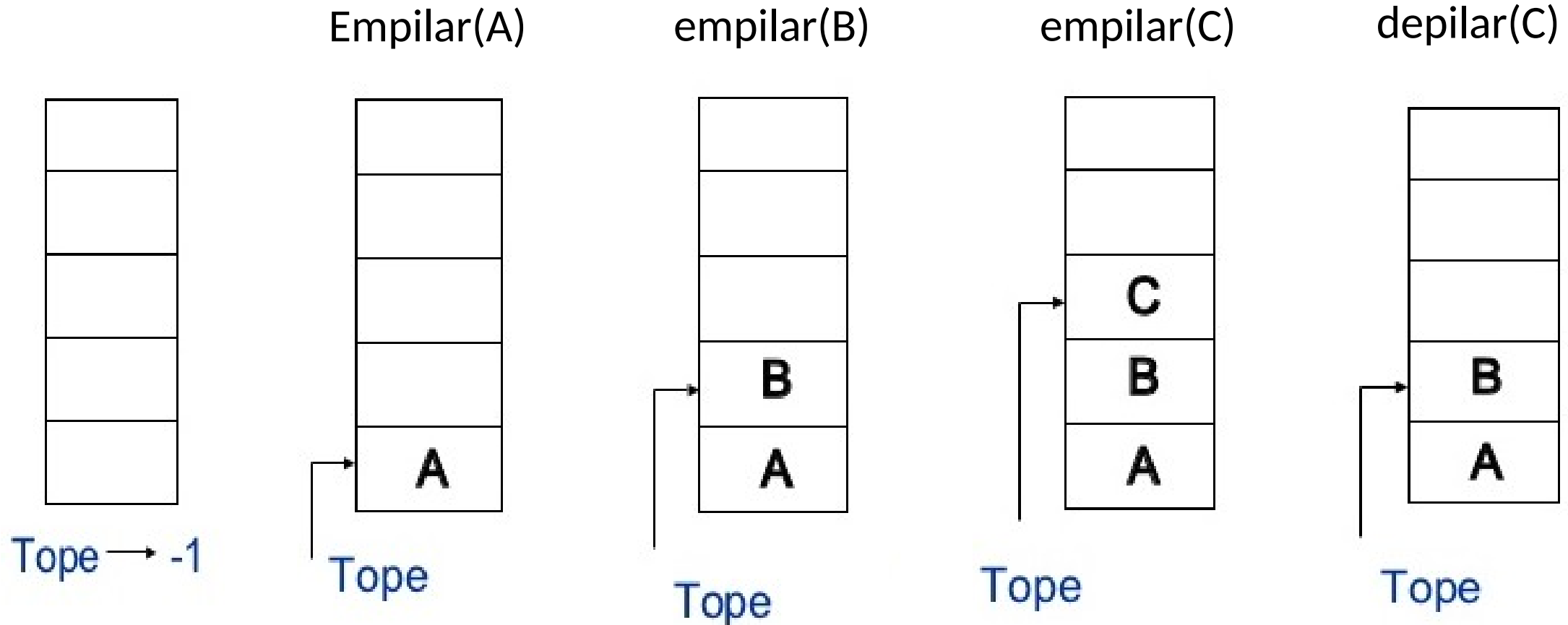
8. Aplicaciones

Principio LIFO. Ultimo en entrar es el primero en salir. Son utiles en::

- **Mecanismo de “Deshacer/Rehacer” CTRL+Z.** Almacenan el historial de cambios en editores de texto y software de diseno.
- **Evaluacion de expresiones matematicas.** Se usa para validar la apertura y cierre de delimitadores y resolver operaciones combinadas.
- **Historia de navegacion Web.** El boton “Atras” del navegador facilita retormar a la ultima pagina visitada apilando las direcciones.

8. Pilas

1. Representación. Pila de caracteres.



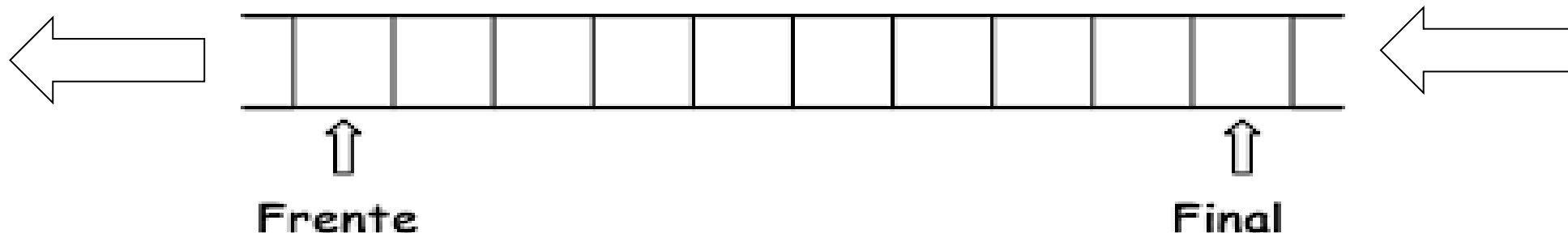
9. Colas

Estructura de acceso restringido

9. Colas

1. Definición. La cola es una estructura lineal de una secuencia de elemento de cierto tipo en donde ***se inserta por un extremo y elimina elementos por el otro extremo.***

- Características
 - Estructura de acceso restringido
 - Es una estructura FIFO (first in first out) primero en entrar primero en salir.
 - Es una colección de datos a los se pueden eliminar por un extremo (frente) y ingresar por el otro extremo (Final).



9. Colas

1. Principalmente se aplica a gestión de recursos.

- **Simulación de situaciones reales:**
 - Simulación de cola de clientes en un supermercado, para determinar el numero de cajeros adecuados para evitar largas colas. Igualmente colas de bancos. Determinar frecuencias adecuadas de buses, metros, etc.
- **Colas de impression.**
 - Para imprimir varios documentos a la vez o la impresora esta ocupado, entonces los trabajos de impresión se almacenan en una cola según el orden de llegada.
- **Sistemas de tiempo compartido.**
 - Los recursos de CPU, memoria, etc., se asigna los procesos que están en cola de espera en el orden que fueron introducidos.

9. Colas

1. Tipos de cola.

- **Cola simple:**

- Estructura línea donde los elementos salen en el mismo orden que arribaron..

- **Colas circular.**

- Representación lógica de cola simple en un arreglo.

- **Cola doble (bicola).**

- Estructura lineal donde los elementos se insertan o eliminan desde cualquier extremo de la cola.

- **Cola de prioridades.**

- Estructura lineal en la cual los elementos se insertan en cualquier posición según su prioridad y se eliminan por el frente de la cola.

9. Aplicaciones

Principio FIFO. Primero en entrar es el primero en salir. Son útiles en

- **Sistemas de impresion:.** Gestionan los files para impresion enviados a una impresora compartida de acuerdo al orden de llegada.
- **Seridorees de aplicaciones.** Almacenan peticiones entrantes de los usuarios para evitar que el sistema colapse y así procesar una por una.
- **Algoritmos de streaming.** En la visualización los videos se almacenan temporalmente en paquetes de datos para asegurar una reproducción continua.

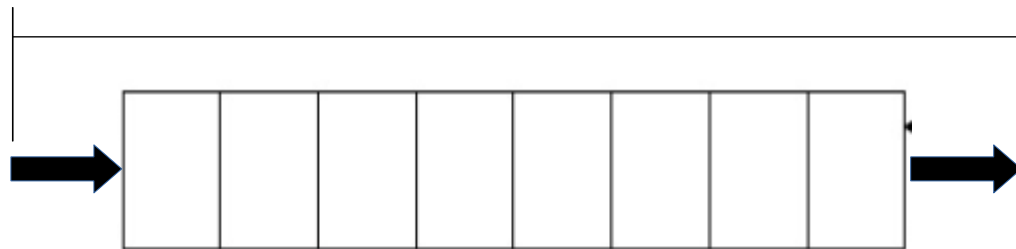
3. Tipos de cola

1. Tipos de cola.

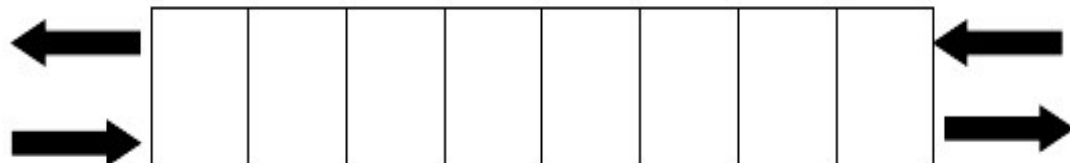
Cola



Cola circular

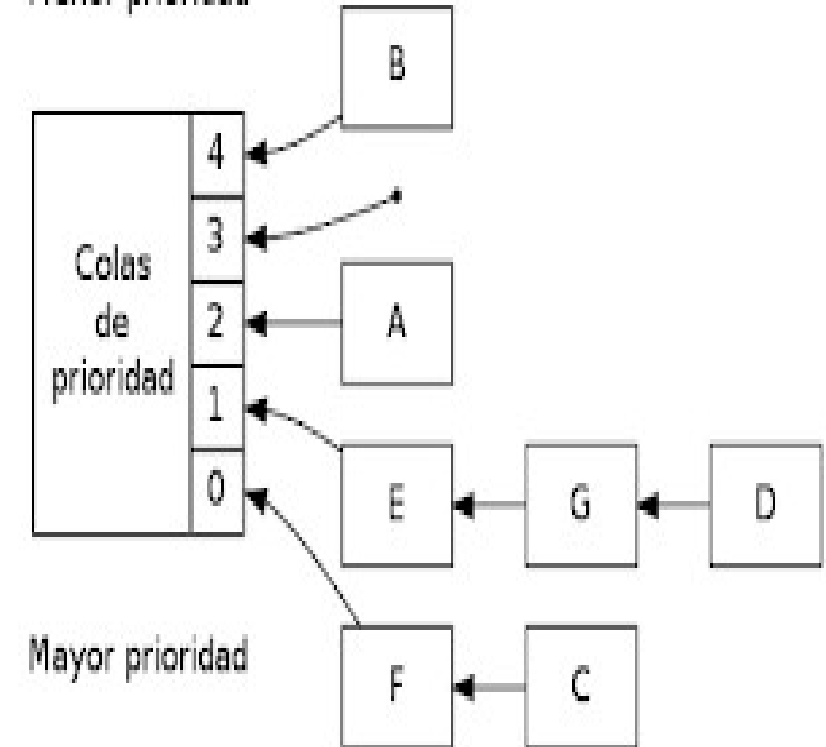


Bicola



Cola de prioridad

Menor prioridad



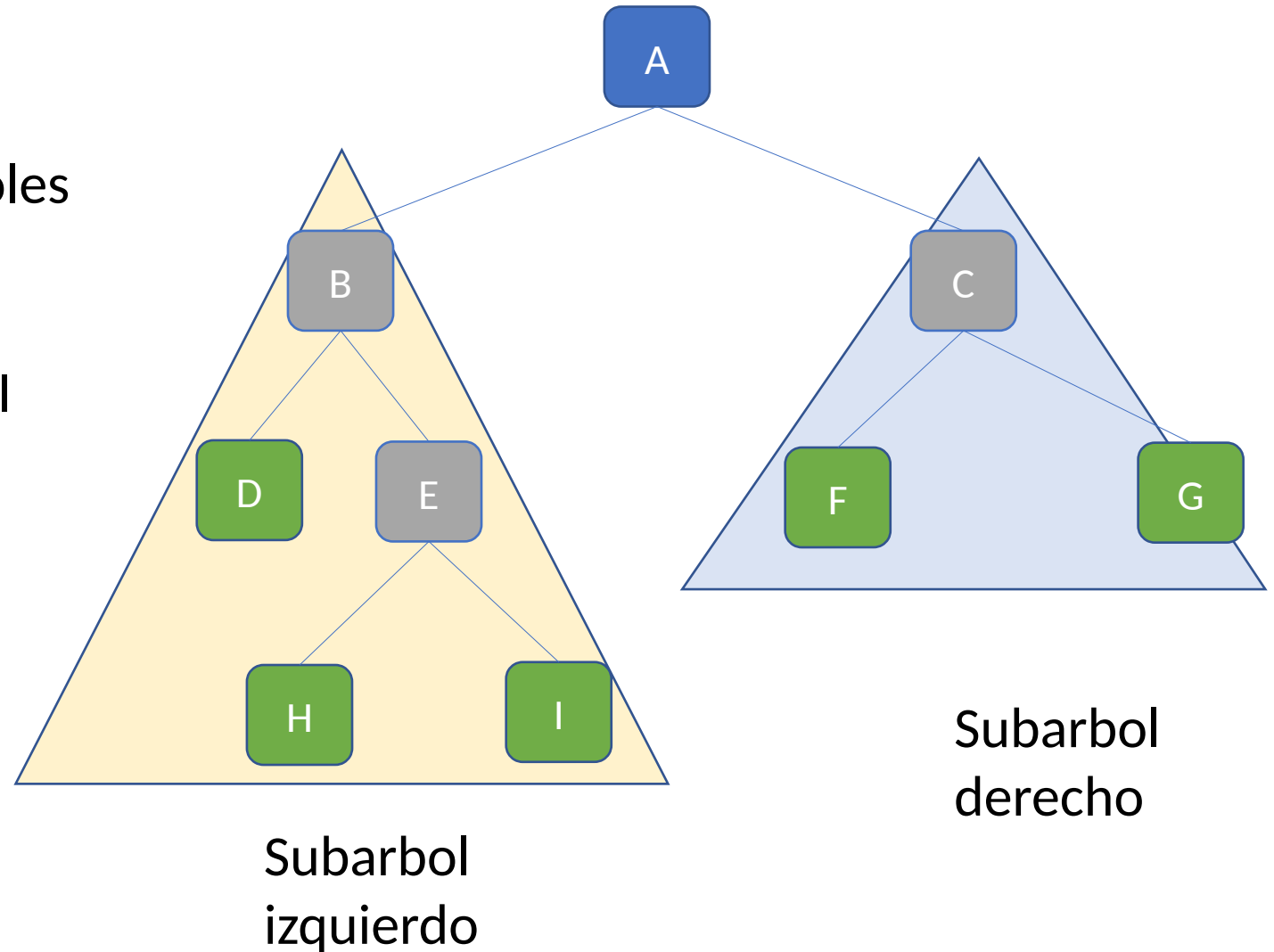
10. Árboles

Estructura de datos no lineales

10. Árboles Binarios

1. Definición

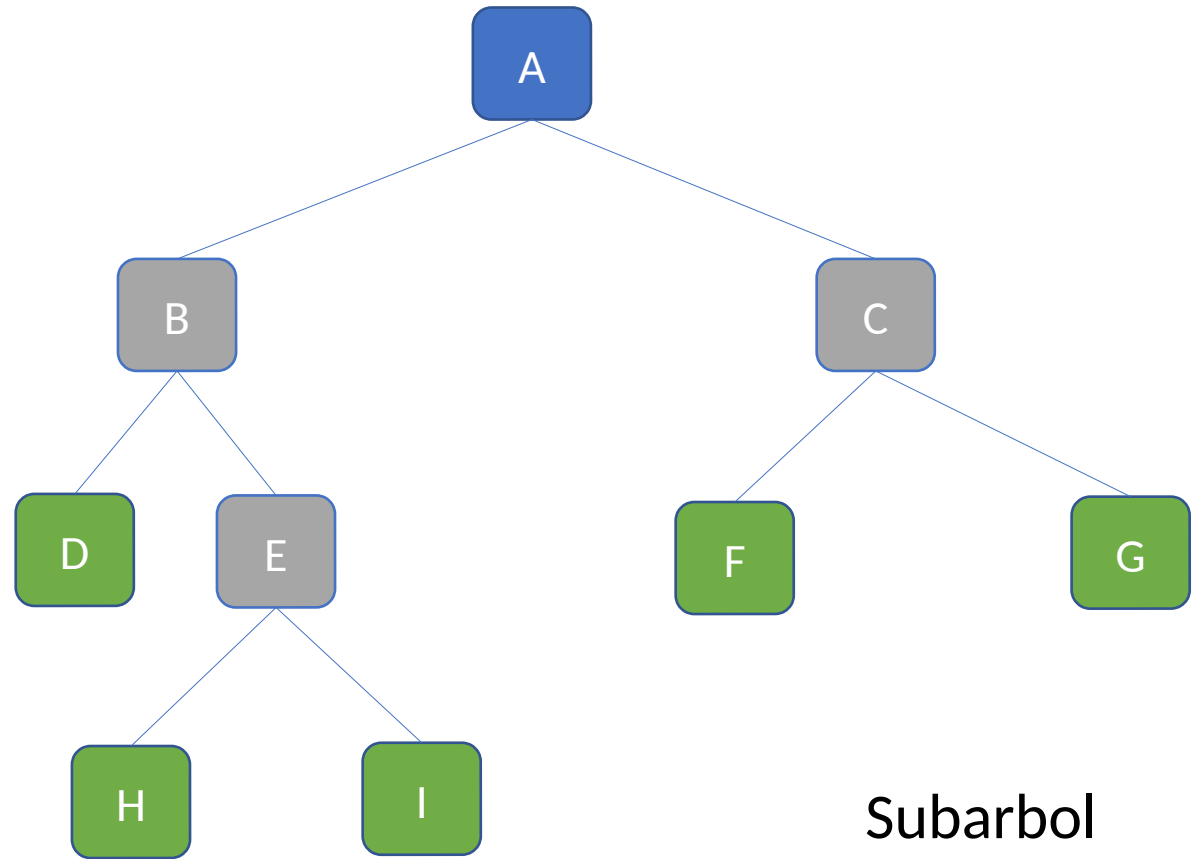
- Arbol de grado 2
 - Cada nodo tiene dos sub arboles (pueden estar vacíos)
 - Nodos internos: ABCE
 - Nodos externos u hoja: DFGHI



10. Árboles Binarios

1. Definición

- Arbol completo
 - Si todo nodo interno (no hoja) tiene dos hijos.



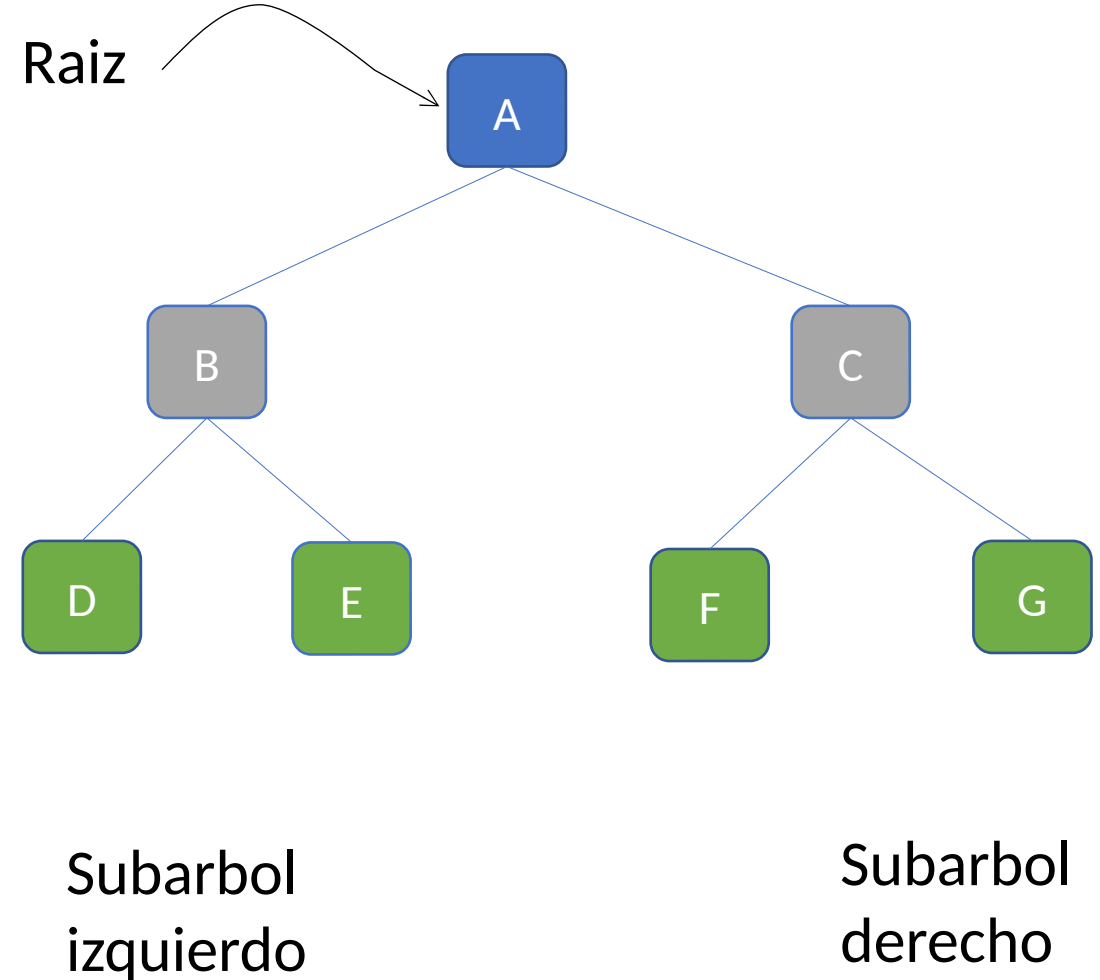
Subarbol
izquierdo

Subarbol
derecho

10. Árboles Binarios

1. Definición

- Arbol lleno
 - Si es un árbol completo y todas sus hojas están en el mismo nivel.



10. Arboles Binarios

El arbol binario esta formado de nodos

- Cada nodo consta de tres campos o partes:
 - Información: data o información del árbol
 - Apuntador al hijo izquierdo
 - Apuntador al hijo derecho

```
REGISTRO NODO
    NODO *hi
    TD info
    NODO *hd
FIN_REGISTRO
```



10. Árboles Binarios

Árboles **BALANCEADOS**

Estos árboles también reciben el nombre de AVL, en honor a sus creadores los matematicos rusos **G.M. Adelson-Velskii** y **E. M. Landis**,

Su contribución principal consistió en presentar algoritmos eficientes de inserción y eliminación considerando el balanceo ene el árbol que su vez repercute en la eficiencia de las bsquedas.

Un arbol balanceado AVL es un ABB donde la altura de los sub arboles izquierdo y derecho se debe diferenciar a lo mas en uno.

10. Arboles Binarios

FACTOR DE EQUILIBRIO (FE):

$$\mathbf{FE} = \mathbf{H_{SAD} - H_{SAI}}$$

H: Altura

SAD: SubArbol Derecho

SAI: SubArbol Izquierdo

10. Arboles Binarios

Tipos de rotación.

Rotación SIMPLE: Involucra DOS NODOS y pueden realizarse por la rama derecha (DD) o rama izquierda (II)

Rotación COMPUESTA : Involucra TRES NODOS y puede realizar por las ramas derecha e izquierda (DI) o por las ramas izquierda y derecha (ID)

REESTRUCTURAR significa rotar los nodos del mismo. La rotación puede ser simple o compuesta.

SIMPLE	:	DD, II
COMPUESTA	:	DI, ID

10. Árboles Binarios

Estructura: Implementación

REGISTRO ANODO

TD val

ANODO *HI

ANODO *HD

ENTERO fe

FIN_REGISTRO

ANODO *N, *N1, *N2

9. Aplicaciones

Arboles. Los arboles organizan la información en forma jerárquica que permiten búsquedas y ordenamientos eficientes, son útiles en:

- **Sistemas de Archivos.** La gestión de archivos en un disco duro se estructura en un árbol de directorios
- **Motores de búsqueda.** Utilizan variantes de árboles (Tries) para autocompletado de texto y sugerencias de búsqueda.

11. Arboles B

Estructura de datos no lineales

11. Árboles B

ARBOLES MULTICAMINO

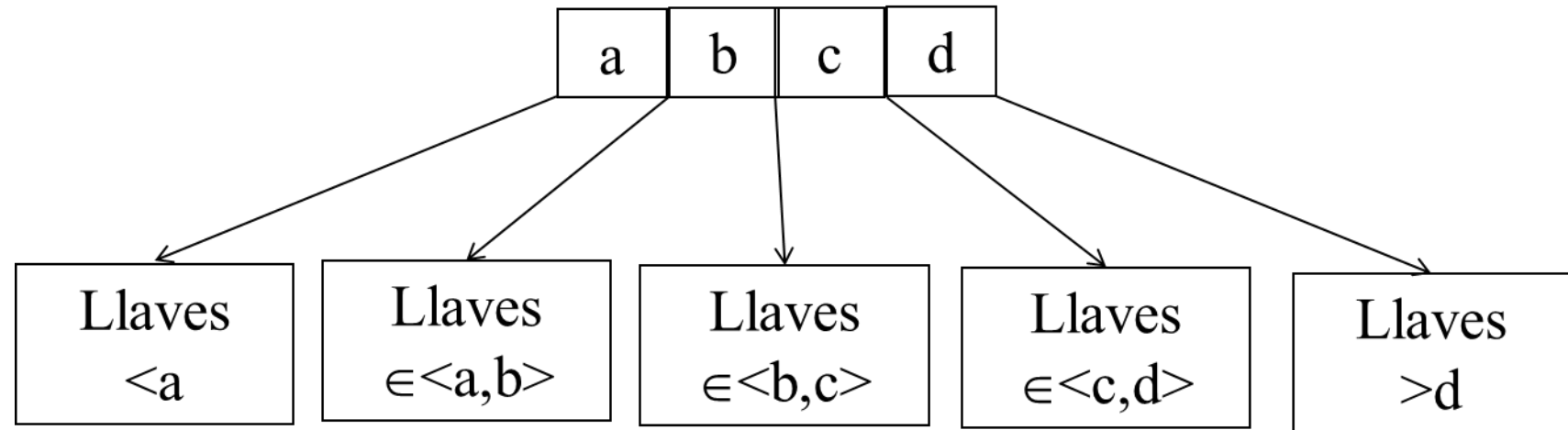
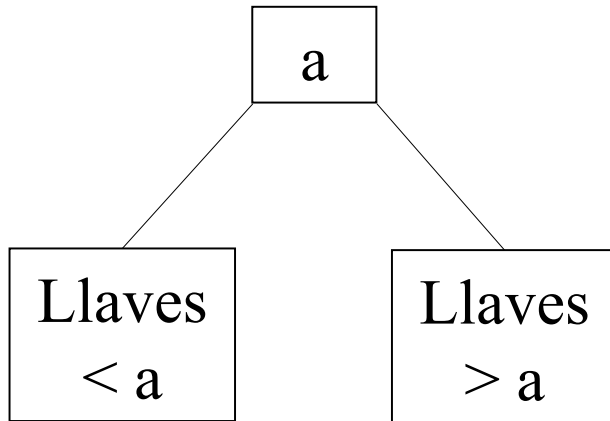
En general se denomina ÁRBOLES MULTICAMINO aquellos árboles de grado mayor a dos. Son muy utilizados en la construcción y mantenimiento de *árboles de búsqueda* de gran cantidad de nodos, usualmente almacenados en MEMORIA SECUNDARIA.

El mas elemental de este tipo de árboles son los Árboles B.

11. Arboles B

Estructura de los arboles de Búsqueda.

- Arbol binario de búsqueda.
- Arbol n-ario de búsqueda.



11. Árboles B

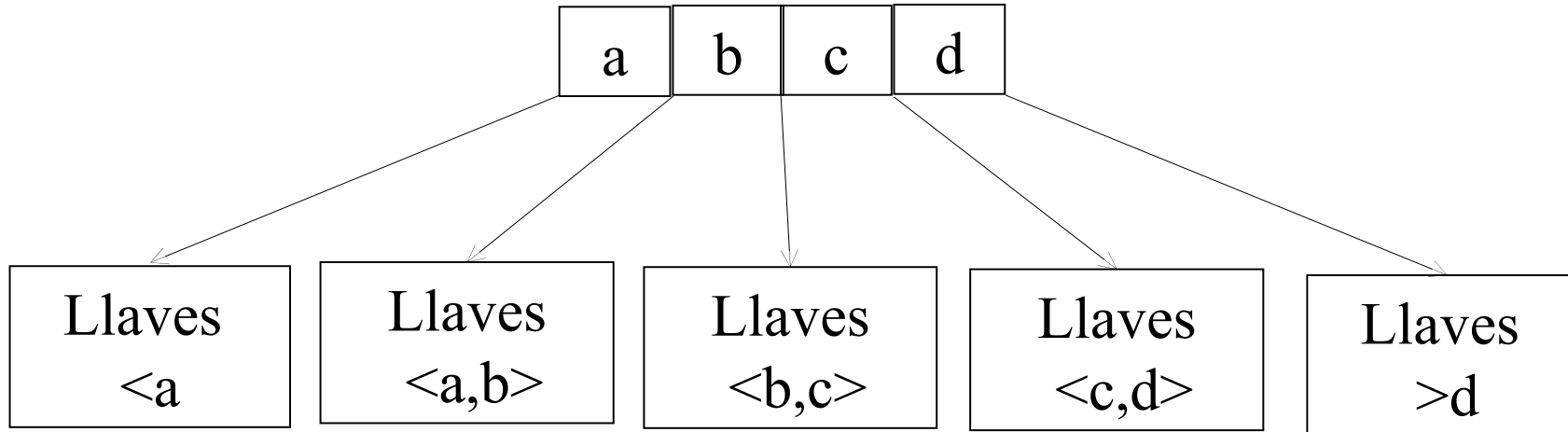
Los ÁRBOLES B, árboles de búsqueda, se basan en el criterio de crecimiento de BAYERN-McCREIGHT y se define de la siguiente forma:

Un árbol B de orden (grado) n es aquel árbol de búsqueda que satisface las siguientes propiedades:

1. Cada página contiene como máximo $2n$ llaves o claves.
2. Cada página contiene como mínimo n llaves, excepto la raíz que puede contener una sola llave.
3. Cada página, es una página de hoja o tiene $m+1$ descendientes. Donde m es el número de llaves en esta página.
4. Todas las páginas de hoja aparecen al mismo nivel.

11. Arboles B

- Las llaves o claves dividen el espacio como en un arbol AVL
- Por ejemplo: $2n = 2 * 2 = 4$
 - Numero maximo por pagina: 4 llaves y 5 descendientes.
 - Numero minimo por pagina: 2 llaves y 3 descendientes.
- Se rastrea el camino de busqueda similar a un arbol binario de busqueda (ABB)



9. Aplicaciones

Arboles B . Los árboles multcamino organizan la información en forma jerárquica que permiten búsquedas y ordenamientos eficientes, son útiles en:

- **Bases de datos.** Emplean árboles B para indexar millones de registros y encontrarlos en milisegundos.

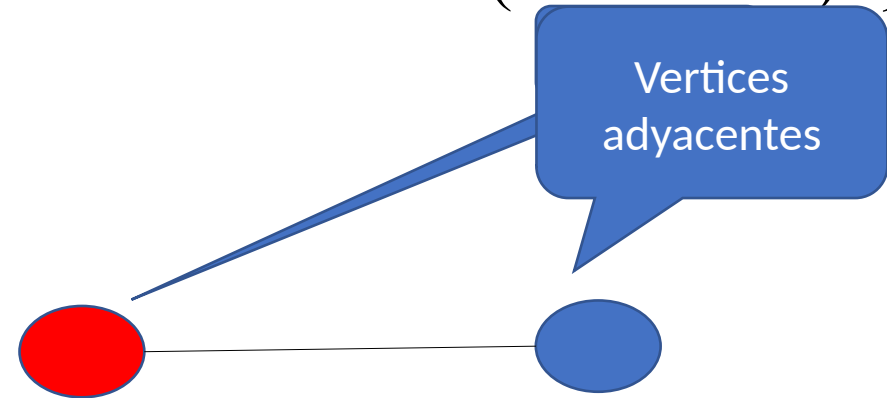
12. Grafos

Estructura de datos no lineales

12. Grafos

Conjuntos de nodos (vértices) y conexiones (aristas), que modelan relaciones complejas (redes sociales, mapas de carreteras, etc).

- **Un grafo G** agrupa entes físicos o conceptuales, es el par $G = (V, A)$, donde V es un conjunto no vacío de **vértices** o **nodos** (entes) y A es un conjunto de **aristas** o **arcos**, que conectan pares de vértices distintos (relaciones). }
- Figura: Arista, **vertices**, **incidencia**



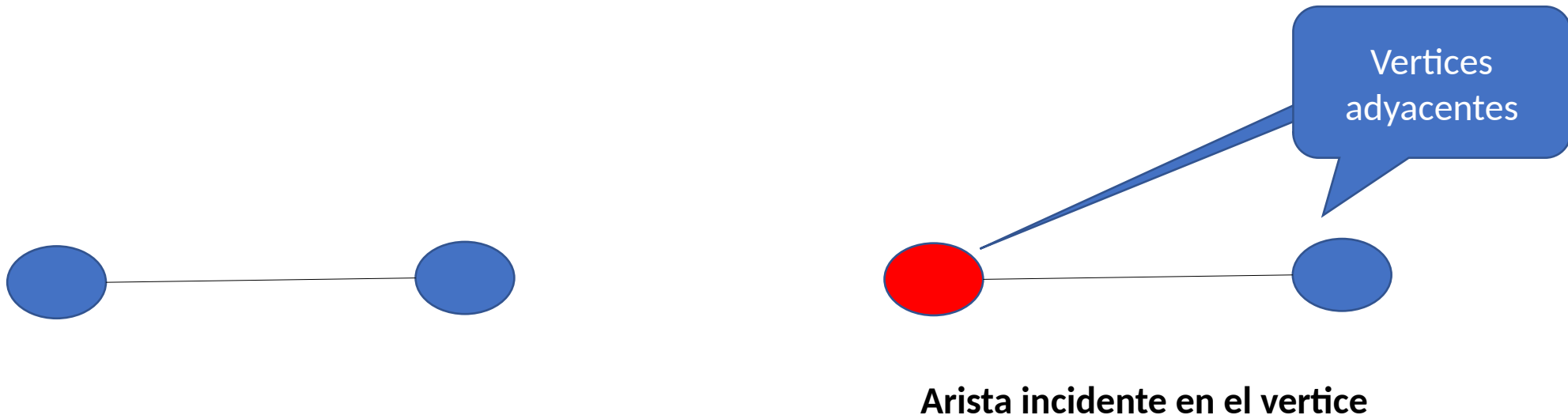
Arista incidente en el vertice

12. Definición

- **Un grafo**

- **Los Grafos.** Enfatizan las relaciones complejas de conexión entre aristas de datos. Los arboles son un caso simple de grafos.
- **Los grafos permiten problemas como.** Menor distancia o costo entre puntos busquedas a través de enlaces en la web; redes eléctricas; cableado de tarjetas impresas; planeación de tareas siguiendo trayectorias;

- **Figura: Arista, vertices, incidencia**



12. Definición

- **Grado de incidencia, del vertice.** Es el numero de aristas que son incidentes en ese vértice.
- **Adyacencia.** En grafos simples se aceptan solo un enlace entre un par de vértices diferentes, no se aceptan lazos o aristas adyacentes en el mismo vértice, ni aristas en paralelo.
- **Trayectoria o camino.** Secuencia de aristas entre vértices adyacentes.
- **Trayectoria simple.** Los vértices y las aristas están presentes solo una vez.
- **Ciclo o circuito.** Trayectoria simple, salvo que el vértice inicial y final son uno solo.
- **Longitud de la trayectoria.** Es el numero de aristas que la componen
- **Grado de un grafo.** Si existe trayectoria entre cualquier par de vértices.

12. Definición

- **Árbol de un grafo.** Subgrafo conectado sin circuitos.
- **Densidad.** Es el promedio de los grados de incidencia de los vértices. Si sumamos los grados de la incidencia se tendrá el valor de $2A$ (2 veces cada arista). Por tanto la densidad es: $2A/V$.
 - Se dice que un grafo es denso si su densidad es proporcional a V . Esto implica que el numero de aristas A será proporcional a V^2 . En caso contrario es un grafo liviano o sparce.
 - El concepto de densidad tiene relación con el tipo de algoritmo y la estructura de datos que se empleara para representar el gafo. Complejidad típicas son: V^2 , y $A\log A$, la primera es mas adecuada a grafos densos y la segunda a grafos livianos.

12. Definición

- **Tipor de grafos:**

- Dirigidos:



- No Dirigidos



- Grafos ponderados. Las aristas $w(A)$ o los vértices tienen asociado un costo o una distancia.
- El árbol de cobertura o Spanning tree de un grafo conectado, con pesos y no dirigido, es un árbol cuyo peso se calcula como la sumatoria de los pesos de las aristas.

12. Definición

- Un grafo $G(V, A)$ es un conjunto no vacío, donde:
 - $V = \{v_1, v_2, \dots, v_M\}$ conjunto de nodos o vértices.
 - $A = \{a_1, a_2, \dots, a_K\}$ es el conjunto de aristas dirigidos (par ordenado de Vértices: (u, v))

G : representa al grafo

- La función $f: A \rightarrow M \times M$ indica los pares de nodos que están relacionados. (Conjunto de aristas o arcos)

12. Representación

- Representación estática:
 - Matriz de adyacencia
- Representación dinámica:
 - Lista de adyacencia.

12. Operaciones

- Recorridos en profundidad:
 - DFS o (Depth First Search)
- Recorridos en anchura:
 - BFS o (Breadth First Search)

9. Aplicaciones

Grafos. Modelan relaciones complejas de red entre sus elementos (vertices) interconectados, son utiles en:

- **Sistemas de Navegacion.** Las esquinas son vertices y las calles son aristas/enlaces. Calculan la ruta mas rapida usando algoritmos de grafos.
- **Redes sociales.** Los usuarios son nodos/vertices y las amistades aristas/conexiones/enlaces. Se usa para sugerir amistades.
- **Etc.**

12. Algoritmos

- **Warshall.** Halla el camino minimo entre todos los vertices
- **Dijkstra.** ¿trayectoria mas corta?
- **Euler.** ¿Paseo euleriano ...?
- **Hamilton.** ¿visitar ciudades ...?
- **Kruskal.** ¿arbol contenido mínimo?
- **Prim** ¿arbol contenido mínimo?
- **Tarjan y Cheriton.** ¿todos contra todos?

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], <http://c.conclase.net/ficheros/>